

**Примітка:** у звіті наведено патерни, розбиті за типами. Після кожного з них наведено приклади з реалізації.

## Патерни створення.

### 1. Factory Method.

- `create_item(item_id)` у `src/items/items.py:978` – фабрична функція, яка за рядковим ідентифікатором ("dull\_sword", "small\_health\_potion" тощо) повертає екземпляр відповідного класу: `MeleeWeapon`, `RangedWeapon`, `Consumable`, `Armor`, `Tool`, `Fish`, `Lantern`, `LightRing`, `GayRing`.
- `create_effect(effect_data)` у `database/effects.py:588` – створює об'єкти ефектів (`PoisonEffect`, `BurnEffect`, `SlowEffect` тощо) зі словника конфігурації.

### 2. Abstract Factory.

- `build_monster_animations(visual_style, size)` у `src/entities/monster_visuals.py` – створює набір анімацій для конкретного візуального стилю монстра.
- `build_attack_controller(attack_profile, config)` у `src/entities/monster_attacks.py` – створює відповідний контролер атак.
- `build_boss_animations(visual_style, size)` – сімейство анімацій для босів.
- `build_boss_attack_controller(attack_profile, config)` - сімейство атак для босів.
- `build_brain(profile, config)` у `src/ai/monster_ai.py:841` – створює AI мозок (`StalkerBrain`, `SkirmisherBrain`, `GuardianBrain`, `ChronosBrain`)  
Профіль ворога потребує узгодженості анімації, атаки та ІІ. Даний патерн гарантує, що всі компоненти будуть сумісні між собою.

### 3. Builder.

- `_create_enemy(x, y, profile)` у `src/core/game.py:1711` - покрокове створення складного об'єкта `Enemy`: вибір профілю, налаштування AI, атаки, анімацій, `patrol points`.
- `Game.__init__()` - складне конструювання ігрового стану: персонаж, мапа, HUD, NPC, інвентарі, міні-ігри, ефекти.

Об'єкти мають багато параметрів. Builder інкапсулює складну логіку створення та дозволяє змінювати представлення (Enemy vs Boss).

#### 4. Prototype.

- `copy.copy(shop_item)` у `src/inventory/system.py:365` - при купівлі товару в магазині створюється копія предмета-зразка.
- `_skill_dicts_from_json / _skill_dicts_to_json` - клонування словників навичок при серіалізації.

#### 5. Singleton.

- `logger = setup_logger()` у `src/core/logger.py:68` - єдиний екземпляр логера для всього застосунку.
- `Gp_database()` - з'єднання з БД (хоча явно не забороняє множинні екземпляри, але використовується як єдина точка доступу).
- `cfg (src/config.py)` - модуль із глобальними конфігураційними змінними (фактичний Singleton на рівні модуля).

Логування, конфігурація та доступ до БД мають бути централізованими, щоб уникнути дублювання ресурсів та неузгодженості стану.

## Структурні патерни.

#### 1. Adapter.

- `DurabilityMixin` у `src/items/items.py:130` - адаптує будь-який клас предмета (Weapon, Armor, Tool) до єдиного інтерфейсу роботи з міцністю.
- `_coerce_int(value, default)` у `src/items/items.py:6` - адаптує типи даних із SQLite (int як float, None) до очікуваних int.
- `_nav_offset(enemy)` у `src/ai/monster_ai.py:129` - адаптує позицію ворога до системи навігації.

SQLite не розрізняє int і float, різні класи предметів мають різні інтерфейси - Adapter забезпечує сумісність без зміни існуючого коду.

#### 2. Bridge.

- У профілях ворогів (`game.py:276-820`) через розділення `visual_style` (візуальне представлення), `ai_profile` (поведінка) та `attack_profile` (атаки). Ці ієрархії незалежні та комбінуються довільним чином.

Візуальний стиль, AI та атаки є доволі далекими. Міст дозволяє змінювати їх незалежно, створюючи десятки комбінацій ворогів.

### 3. Composite.

- StateManager містить словник State об'єктів, делегує їм `handle_event`, `draw`, `update`.
- UI ієрархія: Menu → підменю → віджети (Button, Slider, Dialog).
- Інвентарна система: InventoryManager керує списком Inventory (MAIN\_player\_inventory, ShopInventory, MAIN\_player\_hotbar).

Дозволяє клієнту однаково працювати з окремими елементами та їх групами. Наприклад, `manager.draw(screen)` малює поточний стан незалежно від його складності.

### 4. Decorator.

- `Character.__getattr__` у `src/entities/character.py:736` - динамічно модифікує значення атрибутів `*_damage`, `*_cooldown`, `attack_cooldown` на основі активних пасивок (Mana Flow, Berserker's Rage, Elemental Mastery, Soul Harvest).
- Система ефектів (PoisonEffect, BurnEffect, SlowEffect) динамічно "обгортає" ціль, змінюючи її властивості (`speed_multiplier`, `damage_bonus`, `shield`).
- `DurabilityMixin` додає функціональність міцності до `Weapon`, `Armor`, `Tool`.

Пасивні здібності гравця повинні динамічно впливати на характеристики навичок без зміни їхньої реалізації. Декоратор через `__getattr__` додає цю гнучкість прозоро.

### 5. Facade.

- Клас `App` у `src/app.py:27` - фасад для всього застосунку: ініціалізація вікна, керування станами, аудіо, інвентар, збереження, профілювання.
- `StateManager` - фасад для керування станами (приховує логіку переходів).
- `CollisionSystem` - фасад для обробки колізій між персонажем, ворогами, перешкодами та предметами.

## 6. Flyweight.

- `_ANIMATION_CACHE` та `_FLIPPED_CACHE` у `src/entities/enemy.py:13-14` - кешування анімацій спрайтів за ключем (`sprite_set`, `size`).
- `Item._cached_image / _cached_size` у `src/items/items.py:327-328` - кешування масштабованих зображень предметів.
- `_dir_mask_cache` у `src/app.py:233` - кешування масок напрямку світла.

Безліч кадрів анімацій не створюється наново для кожного ворога - вони спільно використовуються.

## 7. Proxy.

- LOD (Level of Detail) для ворогів у `game.py:2105-2111`: якщо ворог далі ніж 800 пікселів від гравця, оновлення AI та анімацій пропускаються (`active=False`).
- `_is_smeltery_tile` - відкладена перевірка властивостей тайлів карти.

# Поведінкові патерни.

## 1. Chain of Responsibility.

- Ланцюг обробки подій: `App.run()` → `manager.handle_event(event)` → `current_state.handle_event(event)` → специфічні обробники (HUD, інвентар, діалоги).

Події Pygame (клавіші, миша, вікно) проходять через ієрархію обробників, де кожен може обробити або передати далі. Гнучка маршрутизація без жорстких зв'язків.

## 2. Command.

- `use_skill(skill, aim_direction)` у `character.py` - інкапсулює запит на виконання навички.
- `Button.on_click` - колбеки як команди (напр. `on_click=self.open_recipe_menu`).
- `SpawnMenu(on_spawn=self._debug_spawn_enemy)` - команди для дебаг-меню.
- `open_blackjack()`, `open_roulette()`, `open_poker()` з `on_close` колбеками - команди зі зворотним викликом.

Відділяє ініціатора дії (кнопка, хотбар) від виконавця (ігрова логіка).  
Дозволяє параметризувати, ставити в чергу та логувати дії.

### 3. Mediator.

- Клас Game у src/core/game.py:53 виступає центральним посередником між: Character, Enemy (список), NPC, HUD, Map, Projectiles, CollisionSystem, NavGrid, DayNightVisuals, міні-іграми, інвентарями.
- INVENTORY\_manager у src/inventory/inventory\_manager.py - посередник між різними інвентарями (основний, екіпіровка, хотбар, магазин, крафт).

Запобігає прямому зв'язку між десятками компонентів. Всі комунікації проходять через Game.update(), що робить систему керованою.

### 4. Memento.

- SaveManager.save\_game(app, slot\_name) / load\_game(app, slot\_name) у src/core/save\_manager.py:257,396 - зберігає та відновлює повний стан гри (інвентар, характеристики, карту, квести, час доби, переглянуті статті).
- SaveManager.save\_settings / load\_settings - зберігає налаштування (мова, гучність, яскравість, повноекранний режим).

Можливість збереження та завантаження гри - ключова вимога для RPG. Memento інкапсулює стан, не порушуючи інкапсуляції об'єктів.

### 5. Observer.

- logger - класичний Observer: логувальні handler'и (StreamHandler, FileHandler) підписуються на події логера.
- achievement\_manager - спостерігає за подіями: вбивство ворога (unlock("first\_blood")), накопичення грошей (money setter), використання зілля (add\_progress("thirsty")).
- article\_tracker - спостерігає за діями гравця (перше відкриття інвентаря, перше вбивство, перший крафт) і відкриває відповідні статті Wiki.

Дозволяє слабке зв'язування між ігровими подіями та побічними ефектами (досягнення, підказки, логування) без засмічення основної логіки.

## 6. State.

- State / StateManager - головна система станів застосунку: MainMenu, Game, PauseMenu, SettingsMenu тощо. Кожен стан має on\_enter, handle\_event, draw, update.
- AI стани ворогів: "idle", "patrol", "chase", "search", "attack", "retreat", "orbit", "guard", "return", "blink".
- Фази атаки: ATTACK\_PHASE\_IDLE (0), ATTACK\_PHASE\_WIND\_UP (1), ATTACK\_PHASE\_TELEGRAPH (2), ATTACK\_PHASE\_STRIKE (3).
- Boss фази Chronos: Phase 1 (100-70% HP), Phase 2 (70-40%), Phase 3 (40-15%), Phase 4 (15-0%).

Гра має чітко розділені екрани (меню, геймплей, пауза), а вороги - різні поведінки. State патерн робить переходи явними та керованими.

## 7. Strategy.

- AI стратегії: StalkerBrain, SkirmisherBrain, GuardianBrain, ChronosBrain - усі реалізують спільний інтерфейс update(enemy, context).
- Контролери атак: MeleeAttack, BruteAttack, VenomousAttack, ArcanistAttack, TricksterAttack, BomberAttack, PhantomDrain, TitanStomp, CryomancerAttack, ShadowmancerAttack, RevenantAttack, MoltenAttack - різні алгоритми атаки через спільний інтерфейс BaseAttack.
- Стили бою зброї: attack (sword), attack\_mace, attack\_axe, attack\_spear, attack\_war\_hammer

Дозволяє змінювати поведінку ворога та його атаки на льоту (через профіль) без зміни класу Enemy. Додавання нового типу ворога = новий клас Strategy.

## 8. Template Method.

- Effect.update(dt, target) у database/effects.py:30 - визначає скелет: timer += dt → apply(dt, target) → on\_end(target). Підкласи (RegenerationEffect, PoisonEffect тощо) перевизначають apply та on\_end.
- Enemy.update(dt, collision\_system, obstacles, nav\_grid, attack\_context, active) - шаблон: \_tick\_effects → \_tick\_attack\_anim →

\_tick\_attack\_phase → \_move →  
collision\_system.handle\_movement\_and\_collision → \_update\_animation.

- Батьківський Inventory.inventory\_interactions визначає загальну логіку drag-and-drop, а підкласи (MAIN\_player\_inventory, ShopInventory) перевизначають специфічну поведінку.